DLL.cpp implementation:

Search:

Initialize a pointer at location of headptr to traverse the dll
Initialize a variable to count the index of the node currently being checked

While not checking at null: move to next node and compare ssn to ss
If comparison hits, return the index value, otherwise return -1

Insert:

SSN values are inserted in increasing order
If a node already exists with that ssn, return false (do not add), else return true
Else; create a node with (ss, name) and insert into its position

If list is empty, change succ of head and make succ of current nullptr
If middle, change pred of next and succ of previous, insert with appropriate pred and succ
If end, same as empty case but with previous rather than head

Actual code:

Go through and check for a duplicate;
Current pointer at location of headPtr for traversal
While headPtr != null and current ssn <= ss (since dll is ordered), return false if match

Since no match we can make a node with the ssn and name provided

All successful addition cases will increment count and return true

Empty case (headPtr is null)
headPtr becomes pointer to the new node

Beginning case (Current is headPtr) (headPtr isnt null, and the checking loop never satisfied ssn<=ss to advance current, meaning the first node has ssn>ss and the new node must be before it)
New node succ = headPtr, headPtr pred = new node, headPtr = new node
(new node's next is the old head node, old head node predecessor points to new beginning node, headPtr pointer points to new node since it's the first)

End case (if current is nullptr) (similar to beginning, the checking loop satisfied ssn<=ss all the way until current reached null, meaning ss > all ssn and the new node belongs at the end)
This will require having a pointer to the old last node, in this case we can make another traversal pointer and go until the succ is null before inserting the new node.
New node pred points = last node and succ = null. Last node succ = new node.

Middle case (default case) (if a condition hasn't been met yet then do this)
Current is pointing to the first node greater than the new node's ssn, so we need to add the new node in right before current.
New node succ = current, new node pred = current pred, current->pred->succ = new node (the succ pointer of the node before the new node), current pred = new node.

Remove:

Remove the node with provided ss value, return false if none found
Patch up pointers of surrounding nodes, then remove the node once it is not being pointed to

After all successful removals, increment passed count (dcount) and return true

Code:
Check for a duplicate node the same as the insert function, however use ssn < ss (exclusive) to have current land on the dupe. And don't return false when dupe is found.

False case (empty list or no match found)
Return false if current == nullptr (empty list or walked to end) or current ssn != ss (failed the less than check without a match)

first case (current == headPtr)
headPtr = current succ (make second node the head)
If headPtr is not nullptr (avoiding error in case list is now empty), headPtr pred = nullptr
Delete current

last case (current succ == nullptr) (last node hit and passed false check)
Succ of second to last node = nullptr (make it the last)
Delete current

Middle case (default case)
Succ of previous node = current succ, pred of next node = current pred (make next and previous nodes point to eachother)
Delete current

Copy constructor:

Make a current pointer to traverse the original dll

While current doesn't point to nullptr, make a new node with the same data as current

If the new list is currently empty (headPtr == nullptr), make the new node the head
Else, append the new node to the end of the new dll as follows:
Make another pointer "last" to traverse and point to the last node which points to null
Set succ of last node = new node, pred of new node = last (append new to end after last)
Each loop, increment the itemCount and walk current to next in old list

Destructor:
Make current and next pointers for traversal
Current starts at headPtr
While current != null;
next pointer points to current succ, delete current, current = next (delete one at a time)

headPtr = null and itemCount = 0 to make the list empty

Test exe successfully running

```
tyler@DESKTOP-PKVH7HR MINGW64 /c/users/tyler/downloads/proj3
$ ./test.exe
After insertion, we should have 10 20 30 40 50 in order
10      0x2a24a071b00
20      0x2a24a071b60
30      0x2a24a071a40
40      0x2a24a071aa0
50      0x2a24a071bc0
Searching 30 in the list, result should be 2
2
After deletion, we should have 20 30 40 in order
20      0x2a24a071b60
30      0x2a24a071a40
40      0x2a24a071aa0
Testing copy constructor
Contents of the original list
20      0x2a24a071b60
30      0x2a24a071a40
40      0x2a24a071aa0
Contents of the new list, the memory address of the this list must be different from the original list
20      0x2a24a071bc0
30      0x2a24a071c20
40      0x2a24a071c80
```

Project3 implementation done

```
tyler@DESKTOP-PKVH7HR MINGW64 /c/users/tyler/downloads/proj3
$ ./project3.exe 50000-idr
The Number of Valid Insertion: 32769
The Number of Valid Deletion: 3596
The Number of Valid Retrieval: 3638
Number of Items in the List: 29173
elapsed time: 5.52
```